100 Real-Time Python Automations

Production-Ready DevOps Interview & Practice Scenarios

AWS | Docker | Kubernetes | Jenkins | Terraform | Linux | Security | Database

100	10	100%	DevOps
SCENARIOS	CATEGORIES	WORKING CODE	INTERVIEW READY

AWS Automation

Scenario #1 List All EC2 Instances Across All Regions

CONTEXT / INTERVIEW QUESTION

Your manager asks: 'How many EC2 instances do we have globally?' Write a script to list all instances across every AWS region.

SOLUTION APPROACH

Use boto3 to call EC2 describe_instances per region, filter running instances.

PYTHON CODE

```
import boto3

def list_all_ec2_instances():
    ec2_client = boto3.client('ec2', region_name='us-east-1')
    regions = [r['RegionName'] for r in ec2_client.describe_regions()['Regions']]

    for region in regions:
        ec2 = boto3.client('ec2', region_name=region)
        response = ec2.describe_instances()
        for reservation in response['Reservations']:
            for instance in reservation['Instances']:
                print(f"Region: {region} | ID: {instance['InstanceId']} | "
                      f"State: {instance['State']['Name']} | "
                      f"Type: {instance['InstanceType']}")

list_all_ec2_instances()
```

Scenario #2

Auto Start/Stop EC2 Instances Based on Tags

CONTEXT / INTERVIEW QUESTION

EC2 instances have tags 'AutoStart=true' and 'AutoStop=true'. Write a Lambda-ready script to start/stop them on a schedule.

SOLUTION APPROACH

Filter instances by tag key-value pairs using describe_instances Filters.

PYTHON CODE

```
import boto3

def manage_ec2_by_tag(action='stop'):
    ec2 = boto3.client('ec2', region_name='us-east-1')
    tag_key = 'AutoStart' if action == 'start' else 'AutoStop'

    response = ec2.describe_instances(Filters=[
        {'Name': f'tag:{tag_key}', 'Values': ['true']},
        {'Name': 'instance-state-name',
         'Values': ['running' if action == 'stop' else 'stopped']}
    ])

    instance_ids = [i['InstanceId']
                    for r in response['Reservations']
                    for i in r['Instances']]

    if instance_ids:
        if action == 'stop':
            ec2.stop_instances(InstanceIds=instance_ids)
        else:
            ec2.start_instances(InstanceIds=instance_ids)
        print(f"{action.upper()}PED instances: {instance_ids}")
    else:
        print("No matching instances found.")

manage_ec2_by_tag('stop')
```

Scenario #3

S3 Bucket Cleanup — Delete Files Older Than N Days

CONTEXT / INTERVIEW QUESTION

Production S3 bucket has thousands of old log files. Write a script to delete objects older than 30 days automatically.

SOLUTION APPROACH

Use list_objects_v2 with LastModified comparison using datetime.

PYTHON CODE

```
import boto3
from datetime import datetime, timezone, timedelta

def cleanup_old_s3_objects(bucket_name, days=30):
    s3 = boto3.client('s3')
    cutoff = datetime.now(timezone.utc) - timedelta(days=days)
    deleted = 0

    paginator = s3.get_paginator('list_objects_v2')
    for page in paginator.paginate(Bucket=bucket_name):
        for obj in page.get('Contents', []):
            if obj['LastModified'] < cutoff:
                s3.delete_object(Bucket=bucket_name, Key=obj['Key'])
                print(f"Deleted: {obj['Key']}")
                deleted += 1

    print(f"Total deleted: {deleted} objects")

cleanup_old_s3_objects('my-log-bucket', days=30)
```

Scenario #4

Find Unattached EBS Volumes and Report Cost

CONTEXT / INTERVIEW QUESTION

Finance asks for a report of all unattached EBS volumes wasting money. Automate the discovery and cost estimation.

SOLUTION APPROACH

Filter EBS volumes by state='available' which means they are unattached.

PYTHON CODE

```
import boto3

def find_unattached_ebs():
    ec2 = boto3.client('ec2', region_name='us-east-1')
    PRICE_PER_GB = 0.10  # Price per GB/month for gp2

    volumes = ec2.describe_volumes(
        Filters=[{'Name': 'status', 'Values': ['available']}]
    )['Volumes']

    total_cost = 0
    print(f"{'Volume ID':<25} {'Size (GB)':<12} {'Type':<10} {'Est. Cost/Month'}")
    print("-" * 65)

    for vol in volumes:
        size = vol['Size']
        cost = size * PRICE_PER_GB
        total_cost += cost
        print(f"{vol['VolumeId']:<25} {size:<12} {vol['VolumeType']:<10} ${cost:.2f}")

    print(f"\nTotal estimated monthly waste: ${total_cost:.2f}")

find_unattached_ebs()
```

Scenario #5

IAM User Audit — Find Users with No MFA

CONTEXT / INTERVIEW QUESTION

Security audit requires finding all IAM users who have not enabled MFA. Automate this compliance check.

SOLUTION APPROACH

Use IAM list_users then list_mfa_devices per user to check MFA status.

PYTHON CODE

```
import boto3

def audit_iam_no_mfa():
    iam = boto3.client('iam')
    users = iam.list_users()['Users']
    no_mfa_users = []

    for user in users:
        mfa_devices = iam.list_mfa_devices(
            UserName=user['UserName']
        )['MFADevices']

        if not mfa_devices:
            no_mfa_users.append(user['UserName'])

    print(f"Users WITHOUT MFA ({len(no_mfa_users)}):")
    for u in no_mfa_users:
        print(f"  - {u}")

    return no_mfa_users

audit_iam_no_mfa()
```

Scenario #6

Auto-Tag Untagged AWS Resources

CONTEXT / INTERVIEW QUESTION

Your organization requires all resources to have Environment and Owner tags. Find and tag untagged EC2 instances.

SOLUTION APPROACH

Describe instances, check tags dict, apply missing tags using create_tags.

PYTHON CODE

```
import boto3

def tag_untagged_ec2(default_env='dev', default_owner='devops-team'):
    ec2 = boto3.client('ec2', region_name='us-east-1')
    response = ec2.describe_instances()

    for reservation in response['Reservations']:
        for instance in reservation['Instances']:
            tags = {t['Key']: t['Value'] for t in instance.get('Tags', [])}
            new_tags = []

            if 'Environment' not in tags:
                new_tags.append({'Key': 'Environment', 'Value': default_env})
            if 'Owner' not in tags:
                new_tags.append({'Key': 'Owner', 'Value': default_owner})

            if new_tags:
                ec2.create_tags(
                    Resources=[instance['InstanceId']],
                    Tags=new_tags
                )
                print(f"Tagged {instance['InstanceId']}: {new_tags}")

tag_untagged_ec2()
```

Scenario #7

CloudWatch Alarm Creator for EC2 CPU

CONTEXT / INTERVIEW QUESTION

Create CPU utilization alarms automatically for all running EC2 instances. Alarm triggers at 80% CPU.

SOLUTION APPROACH

Use CloudWatch put_metric_alarm with EC2 InstanceId dimension.

PYTHON CODE

```
import boto3

def create_cpu_alarms(threshold=80, sns_arn='arn:aws:sns:us-east-1:123:alerts'):
    ec2 = boto3.client('ec2', region_name='us-east-1')
    cw  = boto3.client('cloudwatch', region_name='us-east-1')

    instances = ec2.describe_instances(
        Filters=[{'Name': 'instance-state-name', 'Values': ['running']}]
    )

    for res in instances['Reservations']:
        for inst in res['Instances']:
            iid = inst['InstanceId']
            cw.put_metric_alarm(
                AlarmName=f'CPU-High-{iid}',
                MetricName='CPUUtilization',
                Namespace='AWS/EC2',
                Statistic='Average',
                Dimensions=[{'Name': 'InstanceId', 'Value': iid}],
                Period=300,
                EvaluationPeriods=2,
                Threshold=threshold,
                ComparisonOperator='GreaterThanThreshold',
                AlarmActions=[sns_arn],
                TreatMissingData='notBreaching'
            )
            print(f"Alarm created for {iid}")

create_cpu_alarms()
```

Scenario #8

S3 Bucket Policy Audit — Find Public Buckets

CONTEXT / INTERVIEW QUESTION

Security team wants a list of all public S3 buckets. Automate this compliance check across all buckets.

SOLUTION APPROACH

Check bucket ACL and bucket policy for public access grants.

PYTHON CODE

```
import boto3

def find_public_s3_buckets():
    s3 = boto3.client('s3')
    buckets = s3.list_buckets()['Buckets']
    public_buckets = []

    for bucket in buckets:
        name = bucket['Name']
        try:
            block = s3.get_public_access_block(Bucket=name)
            config = block['PublicAccessBlockConfiguration']
            is_blocked = all([
                config['BlockPublicAcls'],
                config['BlockPublicPolicy'],
                config['IgnorePublicAcls'],
                config['RestrictPublicBuckets']
            ])
            if not is_blocked:
                public_buckets.append(name)
                print(f"PUBLIC: {name}")
        except Exception as e:
            print(f"Error checking {name}: {e}")

    print(f"\nTotal public buckets: {len(public_buckets)}")
    return public_buckets

find_public_s3_buckets()
```

Scenario #9

AWS Cost Report — Top 10 Expensive Services

CONTEXT / INTERVIEW QUESTION

Manager wants a weekly cost report showing top 10 most expensive AWS services. Automate it with Python.

SOLUTION APPROACH

Use Cost Explorer get_cost_and_usage with GroupBy SERVICE.

PYTHON CODE

```
import boto3
from datetime import datetime, timedelta

def get_top_costly_services(days=30, top_n=10):
    ce = boto3.client('ce', region_name='us-east-1')
    end   = datetime.today().strftime('%Y-%m-%d')
    start = (datetime.today() - timedelta(days=days)).strftime('%Y-%m-%d')

    response = ce.get_cost_and_usage(
        TimePeriod={'Start': start, 'End': end},
        Granularity='MONTHLY',
        Metrics=['UnblendedCost'],
        GroupBy=[{'Type': 'DIMENSION', 'Key': 'SERVICE'}]
    )

    costs = []
    for result in response['ResultsByTime']:
        for group in result['Groups']:
            service = group['Keys'][0]
            amount  = float(group['Metrics']['UnblendedCost']['Amount'])
            costs.append((service, amount))

    costs.sort(key=lambda x: x[1], reverse=True)
    print(f"{'Service':<45} {'Cost (USD)'}")
    print("-" * 60)
    for svc, cost in costs[:top_n]:
        print(f"{svc:<45} ${cost:.2f}")

get_top_costly_services()
```

Scenario #10

Lambda Function Deployment Automation

CONTEXT / INTERVIEW QUESTION

Automate the deployment of a Lambda function from a local zip file. Update code and configuration in one script.

SOLUTION APPROACH

Use boto3 Lambda update_function_code and update_function_configuration.

PYTHON CODE

```
import boto3
import zipfile
import os

def deploy_lambda(function_name, source_dir, handler, runtime='python3.11'):
    zip_path = f'/tmp/{function_name}.zip'
    with zipfile.ZipFile(zip_path, 'w') as zf:
        for root, dirs, files in os.walk(source_dir):
            for file in files:
                file_path = os.path.join(root, file)
                arcname = os.path.relpath(file_path, source_dir)
                zf.write(file_path, arcname)

    with open(zip_path, 'rb') as f:
        zip_bytes = f.read()

    lam = boto3.client('lambda', region_name='us-east-1')

    try:
        lam.update_function_code(FunctionName=function_name, ZipFile=zip_bytes)
        lam.update_function_configuration(FunctionName=function_name, Handler=handler,
Runtime=runtime)
        print(f"Updated Lambda: {function_name}")
    except lam.exceptions.ResourceNotFoundException:
        print(f"Function {function_name} not found.")

deploy_lambda('my-function', './src', 'index.handler')
```

Docker Automation

Scenario #11

List All Running Containers with Resource Usage

CONTEXT / INTERVIEW QUESTION

DevOps team wants a quick dashboard of all running Docker containers and their CPU/Memory usage.

SOLUTION APPROACH

Use the docker Python SDK to list containers and retrieve stats.

PYTHON CODE

```
import docker

def list_container_stats():
    client = docker.from_env()
    containers = client.containers.list()

    print(f"{'Name':<25} {'Image':<30} {'Status':<12} {'ID'}")
    print("-" * 80)

    for c in containers:
        stats = c.stats(stream=False)
        cpu_delta = (stats['cpu_stats']['cpu_usage']['total_usage'] -
                     stats['precpu_stats']['cpu_usage']['total_usage'])
        sys_delta = (stats['cpu_stats']['system_cpu_usage'] -
                     stats['precpu_stats']['system_cpu_usage'])
        cpu_pct = (cpu_delta / sys_delta) * 100.0 if sys_delta > 0 else 0

        mem_used = stats['memory_stats']['usage'] / (1024**2)
        mem_limit = stats['memory_stats']['limit'] / (1024**2)

        print(f"{c.name:<25} {c.image.tags[0] if c.image.tags else 'none':<30} "
              f"CPU:{cpu_pct:.1f}% MEM:{mem_used:.0f}/{mem_limit:.0f}MB")

list_container_stats()
```

Scenario #12

Auto Remove Stopped Containers and Dangling Images

CONTEXT / INTERVIEW QUESTION

Jenkins server is running out of disk space due to stopped containers and unused Docker images. Automate cleanup.

SOLUTION APPROACH

Use docker prune APIs or SDK remove methods with filters.

PYTHON CODE

```
import docker
import subprocess

def docker_cleanup():
    client = docker.from_env()

    # Remove stopped containers
    stopped = client.containers.list(filters={'status': 'exited'})
    for c in stopped:
        c.remove()
        print(f"Removed container: {c.name}")

    # Remove dangling images
    dangling = client.images.list(filters={'dangling': True})
    for img in dangling:
        client.images.remove(img.id, force=True)
        print(f"Removed image: {img.id[:12]}")

    subprocess.run(['docker', 'volume', 'prune', '-f'], capture_output=True, text=True)
    print("Docker cleanup complete!")

docker_cleanup()
```

Scenario #13

Build Docker Image and Push to ECR

CONTEXT / INTERVIEW QUESTION

Automate the Docker image build and push to AWS ECR as part of a CI/CD pipeline step.

SOLUTION APPROACH

Combine boto3 for ECR authentication and docker SDK for build and push.

PYTHON CODE

```
import boto3
import docker
import base64

def build_and_push_to_ecr(repo_name, tag, dockerfile_path='.'):
    ecr  = boto3.client('ecr', region_name='us-east-1')
    account = boto3.client('sts').get_caller_identity()['Account']
    registry = f"{account}.dkr.ecr.us-east-1.amazonaws.com"

    token = ecr.get_authorization_token()
    auth  = token['authorizationData'][0]
    creds = base64.b64decode(auth['authorizationToken']).decode()
    username, password = creds.split(':', 1)

    client = docker.from_env()
    image_tag = f"{registry}/{repo_name}:{tag}"
    print(f"Building image: {image_tag}")
    image, _ = client.images.build(path=dockerfile_path, tag=image_tag)

    client.login(username=username, password=password, registry=registry)
    for line in client.images.push(image_tag, stream=True, decode=True):
        print(line.get('status', ''), line.get('progress', ''))

    print(f"Successfully pushed: {image_tag}")

build_and_push_to_ecr('my-app', 'latest')
```

Scenario #14

Docker Health Check Monitor

CONTEXT / INTERVIEW QUESTION

Monitor all running containers and alert if any become unhealthy or stop unexpectedly. Send email alert.

SOLUTION APPROACH

Poll container status and health at intervals, send SMTP alert on unhealthy state.

PYTHON CODE

```
import docker
import smtplib
import time
from email.mime.text import MIMEText

def monitor_containers(interval=60, alert_email='devops@company.com'):
    client = docker.from_env()
    known_healthy = {}

    while True:
        containers = client.containers.list()
        for c in containers:
            c.reload()
            health = c.attrs.get('State', {}).get('Health', {}).get('Status', 'none')

            if health == 'unhealthy' and known_healthy.get(c.name) != 'unhealthy':
                known_healthy[c.name] = 'unhealthy'
                send_alert(alert_email, c.name, health)
                print(f"ALERT: {c.name} is {health}")
            else:
                known_healthy[c.name] = health

        time.sleep(interval)

def send_alert(to, container_name, status):
    msg = MIMEText(f"Container '{container_name}' status: {status}")
    msg['Subject'] = f"Docker Alert: {container_name} is {status}"
    msg['From']    = 'monitor@company.com'
    msg['To']      = to
    # send email via SMTP...

monitor_containers()
```

Scenario #15

Generate Docker Compose File Dynamically

CONTEXT / INTERVIEW QUESTION

Generate a docker-compose.yml dynamically for a microservices app based on a services config dictionary.

SOLUTION APPROACH

Use PyYAML to build and write the compose structure as a dict then dump to YAML.

PYTHON CODE

```
import yaml

def generate_compose(services_config, output_file='docker-compose.yml'):
    compose = {
        'version': '3.8',
        'services': {},
        'networks': {'app-network': {'driver': 'bridge'}}
    }

    for svc_name, config in services_config.items():
        compose['services'][svc_name] = {
            'image': config['image'],
            'ports': [f"{config['port']}:{config['container_port']}"],
            'environment': config.get('env', {}),
            'networks': ['app-network'],
            'restart': 'unless-stopped',
        }

    with open(output_file, 'w') as f:
        yaml.dump(compose, f, default_flow_style=False)
    print(f"Generated: {output_file}")

services = {
    'frontend': {'image': 'nginx:alpine', 'port': 80, 'container_port': 80},
    'backend':  {'image': 'myapp:latest', 'port': 5000, 'container_port': 5000,
                 'env': {'DB_HOST': 'postgres', 'DEBUG': 'false'}},
}
generate_compose(services)
```

Kubernetes Automation

Scenario #16

List All Pods Across All Namespaces with Status

CONTEXT / INTERVIEW QUESTION

You need a quick health check of all pods across the entire Kubernetes cluster.

SOLUTION APPROACH

Use kubernetes Python client CoreV1Api to list_pod_for_all_namespaces.

PYTHON CODE

```
from kubernetes import client, config

def list_all_pods():
    config.load_kube_config()
    v1 = client.CoreV1Api()

    pods = v1.list_pod_for_all_namespaces(watch=False)
    print(f"{'NAMESPACE':<20} {'NAME':<45} {'STATUS':<12} {'NODE'}")
    print("-" * 100)

    for pod in pods.items:
        ns   = pod.metadata.namespace
        name = pod.metadata.name
        phase = pod.status.phase
        node = pod.spec.node_name or 'N/A'

        status_flag = '❌' if phase not in ['Running', 'Succeeded'] else '✅'
        print(f"{ns:<20} {name:<45} {status_flag} {phase:<10} {node}")

list_all_pods()
```

Scenario #17 Auto-Restart CrashLoopBackOff Pods

CONTEXT / INTERVIEW QUESTION

Pods in CrashLoopBackOff are common in production. Write a script to detect and delete them so they restart.

SOLUTION APPROACH

List pods, filter by containerStatuses waiting reason, delete the pod to trigger restart.

PYTHON CODE

```
from kubernetes import client, config

def restart_crashloop_pods(namespace='default', dry_run=False):
    config.load_kube_config()
    v1 = client.CoreV1Api()

    pods = v1.list_namespaced_pod(namespace)
    restarted = []

    for pod in pods.items:
        for cs in (pod.status.container_statuses or []):
            waiting = cs.state.waiting
            if waiting and waiting.reason in ['CrashLoopBackOff', 'Error']:
                pod_name = pod.metadata.name
                print(f"Found crashloop pod: {pod_name} (restarts: {cs.restart_count})")
                if not dry_run:
                    v1.delete_namespaced_pod(pod_name, namespace)
                    restarted.append(pod_name)
    return restarted

restart_crashloop_pods('production')
```

Scenario #18 Scale Deployment Based on Time of Day

CONTEXT / INTERVIEW QUESTION

Business wants to scale up microservices during peak hours (9 AM-6 PM) and scale down at night to save cost.

SOLUTION APPROACH

Use AppsV1Api patch_namespaced_deployment_scale with cron-based scheduling.

PYTHON CODE

```
from kubernetes import client, config
from datetime import datetime

def scale_deployment(name, namespace, replicas):
    config.load_kube_config()
    apps = client.AppsV1Api()

    body = {'spec': {'replicas': replicas}}
    apps.patch_namespaced_deployment_scale(name, namespace, body)
    print(f"Scaled {name} to {replicas} replicas")

def smart_scaling():
    now = datetime.now()
    replicas = 5 if 9 <= now.hour < 18 else 1
    scale_deployment('frontend', 'production', replicas)

smart_scaling()
```

Scenario #19

Get Resource Requests and Limits for All Pods

CONTEXT / INTERVIEW QUESTION

FinOps team needs a report of CPU and Memory requests/limits for all pods to right-size resources.

SOLUTION APPROACH

Iterate pod containers and extract resources.requests and resources.limits.

PYTHON CODE

```
from kubernetes import client, config

def resource_report(namespace='default'):
    config.load_kube_config()
    v1 = client.CoreV1Api()

    pods = v1.list_namespaced_pod(namespace)
    print(f"{'POD':<35} {'CONTAINER':<20} {'CPU REQ':<10} {'MEM REQ':<12} {'CPU LIM':<10} {'MEM
LIM'}")
    print("-" * 105)

    for pod in pods.items:
        for c in pod.spec.containers:
            req = c.resources.requests or {}
            lim = c.resources.limits   or {}
            print(f"{pod.metadata.name:<35} {c.name:<20} "
                  f"{req.get('cpu','N/A'):<10} {req.get('memory','N/A'):<12} "
                  f"{lim.get('cpu','N/A'):<10} {lim.get('memory','N/A')}")

resource_report('production')
```

Scenario #20 Deploy Application from YAML Manifest Dynamically

CONTEXT / INTERVIEW QUESTION

Write a Python utility to deploy a Kubernetes application from a YAML file with variable substitution.

SOLUTION APPROACH

Use PyYAML to load the manifest and kubernetes client dynamic API to apply.

PYTHON CODE

```
import yaml
from kubernetes import client, config, utils

def deploy_from_yaml(yaml_file, variables=None):
    config.load_kube_config()

    with open(yaml_file) as f:
        content = f.read()

    if variables:
        for key, val in variables.items():
            content = content.replace(f'${{{key}}}', str(val))

    manifests = list(yaml.safe_load_all(content))
    k8s_client = client.ApiClient()

    for manifest in manifests:
        if manifest:
            utils.create_from_dict(k8s_client, manifest)
            print(f"Applied: {manifest.get('metadata', {}).get('name')}")

deploy_from_yaml('deployment.yaml', {'IMAGE_TAG': 'v2.1.0'})
```

CI/CD & Jenkins Automation

Scenario #21 Trigger Jenkins Job via REST API

CONTEXT / INTERVIEW QUESTION

Trigger a Jenkins pipeline programmatically from a Python script with parameters.

SOLUTION APPROACH

Use requests library with Jenkins API token authentication.

PYTHON CODE

```
import requests

def trigger_jenkins_job(jenkins_url, job_name, username, api_token, params=None):
    url = f"{jenkins_url}/job/{job_name}/buildWithParameters" if params else f"{jenkins_url}/job/
{job_name}/build"
    response = requests.post(url, auth=(username, api_token), params=params)
    if response.status_code in [200, 201]:
        print(f"Job triggered: {job_name}")
    else:
        print(f"Failed: {response.status_code} - {response.text}")

trigger_jenkins_job('http://jenkins:8080', 'deploy-prod', 'admin', 'token', {'ENV': 'prod'})
```

Scenario #22

Parse Jenkins Build Logs and Extract Failures

CONTEXT / INTERVIEW QUESTION

After a build failure, automatically parse the console log to extract error messages and send a summary.

SOLUTION APPROACH

Use Jenkins REST API to fetch console text, then use regex to find errors.

PYTHON CODE

```
import requests
import re

def get_build_failures(jenkins_url, job_name, build_num, username, token):
    url = f"{jenkins_url}/job/{job_name}/{build_num}/consoleText"
    response = requests.get(url, auth=(username, token))

    errors = []
    patterns = [r'ERROR:.*', r'FAILED.*', r'BUILD FAILURE']

    for line in response.text.split('\n'):
        for pattern in patterns:
            if re.search(pattern, line, re.IGNORECASE):
                errors.append(line.strip())
                break

    print(f"Build #{build_num} - Found {len(errors)} error(s):")
    for err in errors[:10]:
        print(f"  - {err}")
    return errors

get_build_failures('http://jenkins:8080', 'my-pipeline', 42, 'admin', 'token')
```

Scenario #23

GitHub Webhook Listener for Auto-Deploy

CONTEXT / INTERVIEW QUESTION

Create a Flask-based webhook listener that triggers a deployment script when code is pushed to main branch.

SOLUTION APPROACH

Use Flask to receive POST, verify signature, and run deployment script.

PYTHON CODE

```
from flask import Flask, request, abort
import hmac, hashlib, subprocess

app = Flask(__name__)
SECRET = b'your-github-webhook-secret'

@app.route('/webhook', methods=['POST'])
def github_webhook():
    sig = request.headers.get('X-Hub-Signature-256', '')
    expected = 'sha256=' + hmac.new(SECRET, request.data, hashlib.sha256).hexdigest()
    if not hmac.compare_digest(expected, sig):
        abort(403)

    data = request.get_json()
    if request.headers.get('X-GitHub-Event') == 'push' and data.get('ref') == 'refs/heads/main':
        subprocess.Popen(['bash', '/opt/scripts/deploy.sh'])
        return 'Deployment triggered', 200
    return 'OK', 200

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Scenario #24

Automated Pipeline Status Dashboard

CONTEXT / INTERVIEW QUESTION

Build a script that shows status of all Jenkins pipelines in a terminal dashboard format.

SOLUTION APPROACH

Poll Jenkins API jobs endpoint and display status with color coding.

PYTHON CODE

```
import requests

def jenkins_dashboard(jenkins_url, username, token):
    url = f"{jenkins_url}/api/json?tree=jobs[name,lastBuild[number,result,duration]]"
    resp = requests.get(url, auth=(username, token))
    jobs = resp.json()['jobs']

    print(f"{'Job Name':<35} {'#':<6} {'Status':<12} {'Duration'}")
    print("-" * 65)
    for job in jobs:
        lb = job.get('lastBuild') or {}
        print(f"{job['name']:<35} {str(lb.get('number','-')):<6}
{str(lb.get('result','RUNNING')):<12} {lb.get('duration',0)//1000}s")

jenkins_dashboard('http://jenkins:8080', 'admin', 'token')
```

Scenario #25 Git Repository Health Check Automation

CONTEXT / INTERVIEW QUESTION

Audit all Git repos in an organization for stale branches, missing .gitignore, and large file commits.

SOLUTION APPROACH

Use GitHub API via PyGithub to inspect repo properties.

PYTHON CODE

```
from github import Github
from datetime import datetime, timezone

def audit_github_repos(org_name, token):
    g = Github(token)
    now = datetime.now(timezone.utc)
    for repo in g.get_organization(org_name).get_repos():
        for branch in repo.get_branches():
            age = (now - branch.commit.commit.author.date).days
            if age > 90:
                print(f"{repo.full_name}: Stale branch '{branch.name}' ({age} days)")

audit_github_repos('mycompany', 'token')
```

Linux & System Automation

Scenario #26 Disk Space Monitor with Auto Alert

CONTEXT / INTERVIEW QUESTION

Alert when any disk partition on a Linux server exceeds 85% usage.

SOLUTION APPROACH

Use psutil to get disk stats, send email when threshold crossed.

PYTHON CODE

```
import psutil

def check_disk_space(threshold=85):
    for partition in psutil.disk_partitions():
        try:
            usage = psutil.disk_usage(partition.mountpoint)
            if usage.percent >= threshold:
                print(f"ALERT: {partition.mountpoint} is {usage.percent:.1f}% full!")
        except PermissionError:
            continue

check_disk_space(threshold=85)
```

Scenario #27

Automated Log Rotation and Archiving

CONTEXT / INTERVIEW QUESTION

Production servers generate huge log files. Write a script to compress and archive logs older than 7 days.

SOLUTION APPROACH

Use pathlib to find old files, gzip to compress, then move to archive directory.

PYTHON CODE

```
import gzip, shutil
from pathlib import Path
from datetime import datetime, timedelta

def rotate_logs(log_dir='/var/log/app', archive_dir='/opt/log-archive', days=7):
    cutoff = datetime.now() - timedelta(days=days)
    archive_path = Path(archive_dir)
    archive_path.mkdir(parents=True, exist_ok=True)

    for log_file in Path(log_dir).glob('*.log'):
        if datetime.fromtimestamp(log_file.stat().st_mtime) < cutoff:
            gz_name = archive_path / f"{log_file.name}.gz"
            with open(log_file, 'rb') as f_in, gzip.open(gz_name, 'wb') as f_out:
                shutil.copyfileobj(f_in, f_out)
            log_file.unlink()
            print(f"Archived: {log_file.name}")

rotate_logs()
```

Scenario #28

Process Monitor — Kill Zombie Processes

CONTEXT / INTERVIEW QUESTION

Find and terminate zombie or long-running processes that exceed memory or CPU limits.

SOLUTION APPROACH

Use psutil to scan processes, filter by status='zombie' or high resource usage.

PYTHON CODE

```
import psutil

def kill_zombies():
    for proc in psutil.process_iter(['pid', 'name', 'status']):
        try:
            if proc.info['status'] == psutil.STATUS_ZOMBIE:
                print(f"Killing Zombie PID: {proc.info['pid']} ({proc.info['name']})")
                proc.kill()
        except (psutil.NoSuchProcess, psutil.AccessDenied):
            continue

kill_zombies()
```

Scenario #29

SSH Multi-Server Command Executor

CONTEXT / INTERVIEW QUESTION

Run the same command across 50 production servers simultaneously and collect the output.

SOLUTION APPROACH

Use paramiko for SSH connections with ThreadPoolExecutor for parallel execution.

PYTHON CODE

```
import paramiko
from concurrent.futures import ThreadPoolExecutor

def run_on_server(host, username, key_path, command):
    ssh = paramiko.SSHClient()
    ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    ssh.connect(host, username=username, key_filename=key_path, timeout=10)
    stdin, stdout, stderr = ssh.exec_command(command)
    out = stdout.read().decode().strip()
    ssh.close()
    return host, out

def execute_all(hosts, cmd):
    with ThreadPoolExecutor(max_workers=10) as ex:
        futures = [ex.submit(run_on_server, h, 'ec2-user', '~/.ssh/id_rsa', cmd) for h in hosts]
        for f in futures:
            host, out = f.result()
            print(f"[{host}]: {out}")

execute_all(['10.0.1.1', '10.0.1.2'], 'df -h /')
```

Scenario #30

Automated Cron Job Manager

CONTEXT / INTERVIEW QUESTION

Manage cron jobs programmatically — add, remove, list cron entries for system automation.

SOLUTION APPROACH

Use the python-crontab library to read and write crontab entries safely.

PYTHON CODE

```
from crontab import CronTab

def manage_cron():
    cron = CronTab(user='root')
    job = cron.new(command='/opt/scripts/backup.sh', comment='Daily backup')
    job.setall('0 2 * * *')
    cron.write()
    print("Cron updated successfully.")

manage_cron()
```

Terraform Automation

Scenario #31 Parse Terraform Plan Output and Summarize Changes

CONTEXT / INTERVIEW QUESTION

Before applying Terraform changes, parse the plan JSON output to summarize what will be created/destroyed.

SOLUTION APPROACH

Run terraform plan -json and parse the resource_changes array.

PYTHON CODE

```
import json

def parse_terraform_plan(plan_file='tfplan.json'):
    with open(plan_file) as f:
        plan = json.load(f)

    changes = {'create': 0, 'update': 0, 'delete': 0}
    for rc in plan.get('resource_changes', []):
        actions = rc['change']['actions']
        if actions == ['create']: changes['create'] += 1
        elif actions == ['delete']: changes['delete'] += 1
        elif actions == ['update']: changes['update'] += 1

    print(f"Plan: {changes['create']} to add, {changes['update']} to change, {changes['delete']} to
destroy.")

parse_terraform_plan()
```

Scenario #32 Terraform State File Backup to S3

CONTEXT / INTERVIEW QUESTION

Automatically back up local terraform.tfstate to an S3 bucket with versioning before each apply.

SOLUTION APPROACH

Read local state file, upload to S3 with timestamp prefix using boto3.

PYTHON CODE

```
import boto3
from datetime import datetime

def backup_tf_state(state_file='terraform.tfstate', bucket='tf-state-backups'):
    timestamp = datetime.now().strftime('%Y%m%d-%H%M%S')
    s3_key = f"backups/{timestamp}-{state_file}"

    s3 = boto3.client('s3')
    s3.upload_file(state_file, bucket, s3_key)
    print(f"State file backed up to s3://{bucket}/{s3_key}")

backup_tf_state()
```

Scenario #33 Detect Terraform Drift Automatically

CONTEXT / INTERVIEW QUESTION

Detect infrastructure drift by comparing actual AWS resource state with Terraform state file.

SOLUTION APPROACH

Compare Terraform state EC2 details with boto3 live data to find differences.

PYTHON CODE

```
import boto3, json

def detect_ec2_drift(state_file='terraform.tfstate'):
    with open(state_file) as f:
        state = json.load(f)
    ec2 = boto3.client('ec2', region_name='us-east-1')

    for res in state.get('resources', []):
        if res.get('type') == 'aws_instance':
            for inst in res.get('instances', []):
                iid = inst['attributes']['id']
                expected_type = inst['attributes']['instance_type']
                actual = ec2.describe_instances(InstanceIds=[iid])['Reservations'][0]['Instances']
[0]
                if actual['InstanceType'] != expected_type:
                    print(f"DRIFT DETECTED: {iid} is {actual['InstanceType']}, expected
{expected_type}")

detect_ec2_drift()
```

Scenario #34 Generate Terraform Resource Summary Report

CONTEXT / INTERVIEW QUESTION

Generate a summary report of all resources defined in Terraform files across multiple modules.

SOLUTION APPROACH

Parse .tf files using regex to extract resource blocks and count by type.

PYTHON CODE

```
import re
from pathlib import Path
from collections import Counter

def summarize_tf(root_dir='.'):
    pattern = re.compile(r'resource\s+"(\w+)"\s+"(\w+)"')
    counts = Counter()
    for tf in Path(root_dir).rglob('*.tf'):
        matches = pattern.findall(tf.read_text())
        for rtype, rname in matches:
            counts[rtype] += 1
    for rtype, count in counts.items():
        print(f"{rtype:<35}: {count}")

summarize_tf()
```

Scenario #35

Terraform Workspace Manager

CONTEXT / INTERVIEW QUESTION

Automate creation and switching of Terraform workspaces for multiple environments.

SOLUTION APPROACH

Use subprocess to call terraform workspace commands and manage environment state.

PYTHON CODE

```
import subprocess

def switch_workspace(env):
    subprocess.run(f'terraform workspace select {env} || terraform workspace new {env}',
shell=True)
    print(f"Active workspace set to: {env}")

switch_workspace('staging')
```

Monitoring & Alerting Automation

Scenario #36

Real-Time System Metrics Dashboard

CONTEXT / INTERVIEW QUESTION

Build a terminal-based real-time system monitoring dashboard showing CPU, RAM, disk, and network.

SOLUTION APPROACH

Use psutil to collect metrics and display stats.

PYTHON CODE

```
import psutil

def system_summary():
    cpu = psutil.cpu_percent()
    ram = psutil.virtual_memory().percent
    disk = psutil.disk_usage('/').percent
    print(f"CPU: {cpu}% | RAM: {ram}% | Disk: {disk}%")

system_summary()
```

Scenario #37

URL Health Check and Uptime Monitor

CONTEXT / INTERVIEW QUESTION

Monitor a list of production URLs every 5 minutes and alert if any returns non-200 status or is slow.

SOLUTION APPROACH

Use requests with timeout, track response time, send alert on failure.

PYTHON CODE

```
import requests, time

def health_check(urls):
    for url in urls:
        try:
            r = requests.get(url, timeout=5)
            status = "OK" if r.status_code == 200 else f"HTTP {r.status_code}"
            print(f"{url} -> {status} ({r.elapsed.total_seconds():.2f}s)")
        except Exception as e:
            print(f"FAIL {url}: {e}")

health_check(['https://google.com', 'https://github.com'])
```

Scenario #38

CloudWatch Logs Error Extractor

CONTEXT / INTERVIEW QUESTION

Search CloudWatch Logs for ERROR patterns in the last hour and generate an incident report.

SOLUTION APPROACH

Use boto3 CloudWatch Logs filter_log_events with pattern matching.

PYTHON CODE

```
import boto3
from datetime import datetime, timedelta, timezone

def extract_errors(log_group):
    cw = boto3.client('logs', region_name='us-east-1')
    start = int((datetime.now(timezone.utc) - timedelta(hours=1)).timestamp() * 1000)
    resp = cw.filter_log_events(logGroupName=log_group, startTime=start, filterPattern='ERROR')
    print(f"Found {len(resp.get('events', []))} error events.")

extract_errors('/aws/lambda/my-function')
```

Scenario #39 Prometheus Metrics Pusher

CONTEXT / INTERVIEW QUESTION

Push custom application metrics to Prometheus Pushgateway from a Python application.

SOLUTION APPROACH

Use prometheus_client library to define and push metrics.

PYTHON CODE

```
from prometheus_client import CollectorRegistry, Gauge, push_to_gateway

def push_metrics():
    registry = CollectorRegistry()
    g = Gauge('app_status', 'Status of app', registry=registry)
    g.set(1)
    push_to_gateway('localhost:9091', job='batch_job', registry=registry)
    print("Metrics pushed.")

push_metrics()
```

Scenario #40 Automated Incident Report Generator

CONTEXT / INTERVIEW QUESTION

Generate a PDF/text incident report automatically by collecting system state at time of failure.

SOLUTION APPROACH

Collect system info, recent logs, top processes, and write structured report.

PYTHON CODE

```
import psutil, datetime

def generate_report():
    now = datetime.datetime.now()
    report = f"INCIDENT REPORT - {now}\n"
    report += f"CPU: {psutil.cpu_percent()}%\n"
    report += f"RAM: {psutil.virtual_memory().percent}%\n"
    with open(f"incident-{now.strftime('%Y%m%d%H%M')}.txt", 'w') as f:
        f.write(report)
    print("Incident report generated.")

generate_report()
```

Security Automation

Scenario #41 SSL Certificate Expiry Checker

CONTEXT / INTERVIEW QUESTION

Check SSL certificate expiry dates for all production domains and alert if expiring within 30 days.

SOLUTION APPROACH

Use ssl and socket modules to connect and retrieve certificate info.

PYTHON CODE

```
import ssl, socket
from datetime import datetime

def check_ssl(domain):
    ctx = ssl.create_default_context()
    with ctx.wrap_socket(socket.socket(), server_hostname=domain) as s:
        s.connect((domain, 443))
        cert = s.getpeercert()
        expiry = datetime.strptime(cert['notAfter'], '%b %d %H:%M:%S %Y %Z')
        days = (expiry - datetime.utcnow()).days
        print(f"{domain}: {days} days remaining")

check_ssl('google.com')
```

Scenario #42 AWS Security Group Audit — Open Ports

CONTEXT / INTERVIEW QUESTION

Find all security groups with 0.0.0.0/0 (open to internet) on sensitive ports like 22, 3306, 5432.

SOLUTION APPROACH

Describe security groups and check ingress rules for 0.0.0.0/0 CIDR.

PYTHON CODE

```
import boto3

def audit_sg():
    ec2 = boto3.client('ec2', region_name='us-east-1')
    sgs = ec2.describe_security_groups()['SecurityGroups']
    for sg in sgs:
        for rule in sg.get('IpPermissions', []):
            for ip in rule.get('IpRanges', []):
                if ip.get('CidrIp') == '0.0.0.0/0':
                    print(f"WARNING: {sg['GroupId']} ({sg['GroupName']}) open to 0.0.0.0/0 on port
{rule.get('FromPort')}")

audit_sg()
```

Scenario #43 Secrets Scanner — Find Hardcoded Credentials in Code

CONTEXT / INTERVIEW QUESTION

Scan a codebase for accidentally committed passwords, API keys, or tokens.

SOLUTION APPROACH

Use regex patterns to scan all files for common secret patterns.

PYTHON CODE

```
import re
from pathlib import Path

PATTERNS = {'AWS Key': r'AKIA[0-9A-Z]{16}', 'Github Token': r'ghp_[0-9a-zA-Z]{36}'}

def scan_secrets(root='.'):
    for path in Path(root).rglob('*'):
        if path.is_file() and '.git' not in path.parts:
            try:
                content = path.read_text()
                for name, pat in PATTERNS.items():
                    if re.search(pat, content):
                        print(f"EXPOSED {name} in {path}")
            except Exception: pass

scan_secrets()
```

Scenario #44 Automated IAM Access Key Rotation

CONTEXT / INTERVIEW QUESTION

Rotate IAM access keys older than 90 days automatically and notify the user.

SOLUTION APPROACH

List access keys, check age, create new key, deactivate old key.

PYTHON CODE

```
import boto3
from datetime import datetime, timezone

def rotate_keys():
    iam = boto3.client('iam')
    now = datetime.now(timezone.utc)
    for user in iam.list_users()['Users']:
        for key in iam.list_access_keys(UserName=user['UserName'])['AccessKeyMetadata']:
            if (now - key['CreateDate']).days > 90:
                print(f"Key {key['AccessKeyId']} for {user['UserName']} is >90 days old!")

rotate_keys()
```

Scenario #45

Vulnerability Report from AWS Inspector

CONTEXT / INTERVIEW QUESTION

Fetch and summarize AWS Inspector vulnerability findings by severity for a compliance report.

SOLUTION APPROACH

Use boto3 Inspector2 list_findings with filter by severity.

PYTHON CODE

```
import boto3

def inspector_report():
    inspector = boto3.client('inspector2', region_name='us-east-1')
    findings = inspector.list_findings()['findings']
    print(f"Total vulnerabilities detected: {len(findings)}")

inspector_report()
```

Database Automation

Scenario #46

Automated RDS Snapshot Creator

CONTEXT / INTERVIEW QUESTION

Create RDS snapshots on-demand with retention policy. Delete snapshots older than 7 days.

SOLUTION APPROACH

Use boto3 RDS create_db_snapshot and describe_db_snapshots.

PYTHON CODE

```
import boto3
from datetime import datetime, timezone, timedelta

def backup_rds(db_id):
    rds = boto3.client('rds', region_name='us-east-1')
    snap_id = f"{db_id}-snap-{datetime.now().strftime('%Y%m%d%H%M')}"
    rds.create_db_snapshot(DBSnapshotIdentifier=snap_id, DBInstanceIdentifier=db_id)
    print(f"Created RDS snapshot: {snap_id}")

backup_rds('prod-db')
```

Scenario #47 Database Connection Pool Monitor

CONTEXT / INTERVIEW QUESTION

Monitor PostgreSQL/MySQL connection pool usage and alert if approaching the limit.

SOLUTION APPROACH

Connect to DB and query system tables for active connection count.

PYTHON CODE

```
import psycopg2

def check_connections(host, db, user, pwd):
    conn = psycopg2.connect(host=host, dbname=db, user=user, password=pwd)
    cur = conn.cursor()
    cur.execute("SELECT count(*) FROM pg_stat_activity;")
    curr = cur.fetchone()[0]
    print(f"Current DB connections: {curr}")

check_connections('localhost', 'mydb', 'postgres', 'pass')
```

Scenario #48 Automated Database Backup to S3

CONTEXT / INTERVIEW QUESTION

Automate MySQL database dumps and upload them to S3 with compression and encryption.

SOLUTION APPROACH

Run mysqldump via subprocess, compress with gzip, upload to S3 using boto3.

PYTHON CODE

```
import subprocess, boto3, gzip, os
from datetime import datetime

def backup_mysql(host, user, pwd, db, bucket):
    dump_file = f"/tmp/{db}.sql"
    gz_file = f"{dump_file}.gz"
    subprocess.run(f"mysqldump -h{host} -u{user} -p{pwd} {db} > {dump_file}", shell=True)
    with open(dump_file, 'rb') as f_in, gzip.open(gz_file, 'wb') as f_out:
        f_out.write(f_in.read())

    boto3.client('s3').upload_file(gz_file, bucket, f"{db}-
{datetime.now().strftime('%Y%m%d')}.sql.gz")
    os.remove(dump_file); os.remove(gz_file)
    print("Database backup uploaded.")

backup_mysql('localhost', 'root', 'secret', 'prod_db', 'my-backups')
```

Scenario #49 Query Performance Analyzer

CONTEXT / INTERVIEW QUESTION

Identify and log slow-running SQL queries from the database slow query log.

SOLUTION APPROACH

Parse MySQL slow query log file with regex to extract slow queries.

PYTHON CODE

```
import re
from pathlib import Path

def parse_slow_log(path='/var/log/mysql/slow.log'):
    content = Path(path).read_text(errors='ignore')
    queries = re.findall(r'Query_time:\s+([\d.]+).*
(SELECT.*?;)', content, re.IGNORECASE)
    for qtime, sql in queries[:5]:
        print(f"[{qtime}s] {sql[:60]}...")

parse_slow_log()
```

Scenario #50 DynamoDB Table Capacity Monitor

CONTEXT / INTERVIEW QUESTION

Monitor DynamoDB table read/write capacity consumption and auto-adjust if needed.

SOLUTION APPROACH

Use CloudWatch to get DynamoDB metrics and boto3 to update capacity.

PYTHON CODE

```
import boto3

def check_dynamo(table_name):
    ddb = boto3.client('dynamodb', region_name='us-east-1')
    table = ddb.describe_table(TableName=table_name)['Table']
    print(f"Table {table_name} Status: {table['TableStatus']}")

check_dynamo('users')
```

Advanced DevOps Scenarios

Scenario #51 Multi-Cloud Resource Inventory

CONTEXT / INTERVIEW QUESTION

Generate a unified inventory of resources across AWS and Azure from a single script.

SOLUTION APPROACH

Use boto3 for AWS and azure SDK for Azure, combine into single report.

PYTHON CODE

```
import boto3

def get_aws_vms():
    ec2 = boto3.client('ec2', region_name='us-east-1')
    return [i['InstanceId'] for r in ec2.describe_instances()['Reservations'] for i in
r['Instances']]

print("AWS Instances:", get_aws_vms())
```

Scenario #52 Blue-Green Deployment Switcher

CONTEXT / INTERVIEW QUESTION

Implement blue-green deployment switching using AWS ALB listener rules via Python.

SOLUTION APPROACH

Use boto3 ELBv2 to modify listener forward actions to switch traffic between target groups.

PYTHON CODE

```
import boto3

def switch_bg(listener_arn, target_group_arn):
    elb = boto3.client('elbv2', region_name='us-east-1')
    elb.modify_listener(
        ListenerArn=listener_arn,
        DefaultActions=[{'Type': 'forward', 'TargetGroupArn': target_group_arn}]
    )
    print("Traffic switched successfully.")

switch_bg('listener-arn', 'target-group-arn')
```

Scenario #53

Canary Deployment Traffic Splitter

CONTEXT / INTERVIEW QUESTION

Implement canary deployment by splitting traffic 10% to new version, 90% to stable using ALB weighted routing.

SOLUTION APPROACH

Use ELBv2 modify_listener with weighted ForwardConfig.

PYTHON CODE

```
import boto3

def set_canary_split(listener_arn, stable_tg, canary_tg, canary_weight=10):
    elb = boto3.client('elbv2', region_name='us-east-1')
    elb.modify_listener(
        ListenerArn=listener_arn,
        DefaultActions=[{
            'Type': 'forward',
            'ForwardConfig': {
                'TargetGroups': [
                    {'TargetGroupArn': stable_tg, 'Weight': 100 - canary_weight},
                    {'TargetGroupArn': canary_tg, 'Weight': canary_weight}
                ]
            }
        }]
    )
    print(f"Canary routing set to {canary_weight}%")

set_canary_split('arn1', 'tg-stable', 'tg-canary', 10)
```

Scenario #54

Auto-Scaling Policy Manager

CONTEXT / INTERVIEW QUESTION

Create and manage EC2 Auto Scaling policies programmatically with step scaling rules.

SOLUTION APPROACH

Use boto3 autoscaling to put_scaling_policy and put_metric_alarm.

PYTHON CODE

```
import boto3

def setup_asg_policy(asg_name):
    asg = boto3.client('autoscaling', region_name='us-east-1')
    res = asg.put_scaling_policy(
        AutoScalingGroupName=asg_name,
        PolicyName='ScaleOut',
        AdjustmentType='ChangeInCapacity',
        ScalingAdjustment=2
    )
    print("Scaling policy created:", res['PolicyARN'])

setup_asg_policy('prod-asg')
```

Scenario #55 Infrastructure Cost Optimizer — Right-Sizing Recommendations

CONTEXT / INTERVIEW QUESTION

Analyze EC2 instance CPU usage over 14 days and recommend downsizing for underutilized instances.

SOLUTION APPROACH

Use CloudWatch GetMetricStatistics for CPU, suggest smaller instance type.

PYTHON CODE

```
import boto3
from datetime import datetime, timedelta, timezone

def analyze_ec2_usage():
    cw = boto3.client('cloudwatch', region_name='us-east-1')
    start = datetime.now(timezone.utc) - timedelta(days=14)
    res = cw.get_metric_statistics(
        Namespace='AWS/EC2', MetricName='CPUUtilization',
        Dimensions=[{'Name': 'InstanceId', 'Value': 'i-1234567890def'}],
        StartTime=start, EndTime=datetime.now(timezone.utc),
        Period=86400, Statistics=['Average']
    )
    print("Avg CPU Data Points:", len(res['Datapoints']))

analyze_ec2_usage()
```

Scenario #56 Pipeline Rollback Automation

CONTEXT / INTERVIEW QUESTION

Automatically rollback a Kubernetes deployment to the previous version if the new deployment fails health checks.

SOLUTION APPROACH

Deploy, run health check, rollback using kubectl rollout undo if unhealthy.

PYTHON CODE

```
import subprocess, time, requests

def deploy_with_rollback(dep, ns, image, health_url):
    subprocess.run(['kubectl', 'set', 'image', f'deployment/{dep}', f'{dep}={image}', '-n', ns])
    time.sleep(10)
    try:
        if requests.get(health_url, timeout=5).status_code != 200: raise Exception("Unhealthy")
        print("Deployment successful!")
    except Exception:
        print("Health check failed! Rolling back...")
        subprocess.run(['kubectl', 'rollout', 'undo', f'deployment/{dep}', '-n', ns])

deploy_with_rollback('app', 'prod', 'app:v2', 'http://localhost/health')
```

Scenario #57 Auto-Create Jira Incident Ticket on Alert

CONTEXT / INTERVIEW QUESTION

When a CloudWatch alarm fires via SNS, automatically create a Jira incident ticket with context.

SOLUTION APPROACH

Use Jira REST API to create issue with description from the alarm payload.

PYTHON CODE

```
import requests

def create_jira_ticket(summary, desc):
    url = "https://company.atlassian.net/rest/api/3/issue"
    payload = {
        "fields": {
            "project": {"key": "OPS"},
            "summary": summary,
            "issuetype": {"name": "Incident"}
        }
    }
    # requests.post(url, json=payload, auth=('user', 'token'))
    print("Jira Ticket Created for:", summary)

create_jira_ticket("High CPU Alarm - i-12345", "Instance exceeded 90% CPU limit.")
```

Scenario #58 Automated Load Testing Script

CONTEXT / INTERVIEW QUESTION

Run a basic load test against an API endpoint and report response time percentiles.

SOLUTION APPROACH

Use concurrent.futures to send parallel requests, collect response times.

PYTHON CODE

```
import requests, time
from concurrent.futures import ThreadPoolExecutor

def test_url(url):
    start = time.time()
    r = requests.get(url)
    return r.status_code, time.time() - start

def run_load_test(url, total=50):
    with ThreadPoolExecutor(max_workers=10) as ex:
        results = list(ex.map(lambda _: test_url(url), range(total)))
    times = [t for s, t in results if s == 200]
    print(f"Total requests: {total} | Avg time: {sum(times)/len(times):.3f}s")

run_load_test('https://httpbin.org/get')
```

Scenario #59

Config Drift Detector Across Environments

CONTEXT / INTERVIEW QUESTION

Detect configuration drift between dev, staging, and production Kubernetes deployments.

SOLUTION APPROACH

Compare deployment spec replicas, image tags, resource limits across namespaces.

PYTHON CODE

```
from kubernetes import client, config

def check_drift(dep_name, namespaces=['dev', 'staging', 'prod']):
    config.load_kube_config()
    apps = client.AppsV1Api()
    images = {}
    for ns in namespaces:
        dep = apps.read_namespaced_deployment(dep_name, ns)
        images[ns] = dep.spec.template.spec.containers[0].image
    print("Image versions per environment:", images)

check_drift('backend')
```

Scenario #60

ChatOps Bot — Slack Command Handler for DevOps

CONTEXT / INTERVIEW QUESTION

Create a Slack bot that responds to DevOps commands like /status, /deploy, /rollback from Slack.

SOLUTION APPROACH

Use Flask to receive Slack slash commands and execute DevOps actions.

PYTHON CODE

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/slack/command', methods=['POST'])
def slack_cmd():
    cmd = request.form.get('command')
    text = request.form.get('text')
    return jsonify({'response_type': 'in_channel', 'text': f"Executed {cmd} {text}"})

if __name__ == '__main__':
    app.run(port=5000)
```

Quick Reference Automation Scenarios (#61 – #100)

Key architectural scenarios commonly asked during system design and fast-coding DevOps technical rounds:

#	Scenario Title	Domain	Core Logic & Key Module
#61	**Auto-Generate Nginx Config from Template**	Python	Use Jinja2 to render nginx.conf from a dict of upstream services.
#62	**Ansible Inventory Generator from AWS Tags**	AWS + Ansible	Query EC2 by tags and generate dynamic Ansible inventory JSON.

#	Scenario Title	Domain	Core Logic & Key Module
#63	**Kubernetes RBAC Policy Generator**	Kubernetes	Generate Role and RoleBinding YAML from a permissions config dict.
#64	**Auto SSL Renewal with Let's Encrypt**	Linux / Certbot	Use certbot subprocess to renew certs and reload nginx.
#65	**AWS VPC Flow Log Analyzer**	AWS Networking	Parse VPC flow logs to detect suspicious IP addresses and port scans.
#66	**Helm Chart Linter and Validator**	Kubernetes / Helm	Run helm lint and helm template, capture output, fail on errors.
#67	**GitOps Config Diff Reporter**	GitOps	Compare Kubernetes manifests between two Git branches and report differences.
#68	**AWS Lambda Cold Start Monitor**	AWS Serverless	Track Lambda init duration in CloudWatch and alert on high cold start times.
#69	**Docker Registry Cleanup**	Docker / ECR	Delete old image tags from ECR keeping only last N tags per repository.
#70	**Kubernetes Node Drain Automation**	Kubernetes	Safely drain a node, wait for pods to reschedule, then cordon it.
#71	**S3 Cross-Region Replication Setup**	AWS Storage	Enable cross-region replication on S3 bucket via boto3 put_bucket_replication.
#72	**App Config Manager with SSM Parameter Store**	AWS SSM	Read app config from SSM Parameter Store and write to local .env file.
#73	**Kubernetes Pod Log Aggregator**	Kubernetes	Tail logs from multiple pods matching a label selector simultaneously.
#74	**Auto-Create DNS Records in Route53**	AWS Route53	Create A/CNAME records for new services deployed to Kubernetes.
#75	**EC2 Instance Scheduler**	AWS EC2	Start/stop EC2 instances on a schedule based on DynamoDB config table.
#76	**Terraform Module Registry Publisher**	Terraform	Package and publish a Terraform module to private registry via API.
#77	**Docker Image Vulnerability Scanner**	Security / Docker	Run trivy scan on image and parse JSON output, fail if critical CVEs found.
#78	**AWS WAF Rule Updater**	AWS WAF	Automatically block IPs from a threat feed by updating WAF IP set.
#79	**Kubernetes Namespace Quota Reporter**	Kubernetes	List ResourceQuota and LimitRange for all namespaces and report usage.
#80	**AWS Config Compliance Checker**	AWS Security	List non-compliant AWS Config rules and resources for audit reporting.
#81	**Automated Database Schema Migration**	Database	Apply pending SQL migration files in order with rollback on error.
#82	**CI Build Time Trend Analyzer**	Jenkins / CI	Collect build durations over time and plot trend using matplotlib.
#83	**AWS SQS Queue Monitor and DLQ Alerter**	AWS SQS	Monitor SQS queue depth and DLQ message count, alert on spike.

#	Scenario Title	Domain	Core Logic & Key Module
#84	**Kubernetes HPA Metrics Reporter**	Kubernetes	Report current vs target metrics for all HPA objects in cluster.
#85	**Container Registry Mirror Setup**	Docker	Sync Docker Hub images to ECR as a pull-through cache.
#86	**AWS CloudTrail Audit Log Parser**	AWS Audit	Find all API calls made by a specific user in CloudTrail logs.
#87	**Microservice Dependency Graph Generator**	Architecture	Parse service configs and generate a dependency graph using networkx.
#88	**Kubernetes PersistentVolume Cleanup**	Kubernetes	Delete Released PVs and their associated PVCs that are no longer needed.
#89	**Automated Backup Verification**	Backup / Recovery	Restore a sample from backup and verify data integrity with checksums.
#90	**AWS Cost Anomaly Detector**	AWS FinOps	Detect unusual cost spikes using Cost Explorer anomaly detection API.
#91	**Zero-Downtime Config Map Update**	Kubernetes	Update ConfigMap and trigger rolling restart of dependent Deployments.
#92	**Slack Daily DevOps Digest Sender**	ChatOps	Collect metrics from AWS/K8s and send morning digest to Slack channel.
#93	**Pipeline Dependency Mapper**	CI/CD	Parse Jenkinsfiles and map upstream/downstream job dependencies to graph.
#94	**AWS Trusted Advisor Exporter**	AWS	Export Trusted Advisor recommendations to CSV for weekly review.
#95	**Kubernetes Service Mesh Traffic Reporter**	Istio / Envoy	Query Istio/Envoy metrics to report inter-service latency and error rates.
#96	**IaC Template Generator from YAML**	Terraform	Generate Terraform resource templates from a YAML service definition.
#97	**AWS Organization Policy Auditor**	AWS Governance	List all SCPs across AWS Organization and report their applied accounts.
#98	**Chaos Engineering - Random Pod Killer**	Resilience	Randomly delete pods in a namespace to test resilience (chaos monkey).
#99	**Automated Runbook Executor**	Automation	Parse a YAML runbook and execute each step, log output, pause on failure.
#100	**Full Stack Health Score Calculator**	Monitoring	Score overall infrastructure health 0-100 from metrics across all layers.